

Object Oriented Programming (Comp 431)

Chapter Two: Basics in Java Programming

Abey B. (MSc.)

American College of Technology(ACT)
Faculty of Computing
Department of Computer Science

January 16, 2023

- Basic Concepts
 - ▶ Variable types and identifiers
 - ▶ Number types, strings, constants
 - ▶ Operators and operator precedence
 - ▶ Type Conversion/ Casting

Basic Structure of Java Program

Java program involves the following section

- Documentation Section
- Package Statement
- Import Statements
- Interface Statement
- Class Definition
- Main Method Class
 - ▶ Main Method Definition

Basic Structure of Java Program

Documentation Section

- The documentation section is an **important section but optional** for a Java program.
- It includes **basic information about a Java program**.
- The information includes
 - ▶ The author's name
 - ▶ Date of creation
 - ▶ Version, program name
 - ▶ Company name
 - ▶ description of the program.
- The Java compiler ignores the statements during the execution of the program.
- To write the statements in the documentation section, we use comments.

Basic Structure of Java Program

Documentation Section

- The comments may be **single-line, multi-line, and documentation comments**.

- ▶ **Single-line Comment:**

- ★ It starts with a pair of forwarding slash (//).

For example:

```
//First Java Program
```

- ▶ **Multi-line Comment:**

- ★ It starts with a /* and ends with */. We write between these two symbols.

For example:

```
/*It is an example of  
multiline comment*/
```

- ▶ **Documentation Comment:**

- ★ It starts with the delimiter (/**) and ends with */.

For example:

```
/**It is an example of documentation comment*/
```

Basic Structure of Java Program

Package Declaration

- The package declaration is **optional**.
- It must be defined before any class and interface declaration.
- You can create a package with any name.
- A package is **a group of classes** that are defined **by a name**.
- if you want to declare many classes **within one element**, then you can declare it within a **package**.
- It is declared as:
package package_ name;

Basic Structure of Java Program

Import Statements

- The package contains the many **predefined classes and interfaces**.
- If we want to use **any class** of a **particular package**, we need to **import that class**.
- We use the **import keyword** to import the class.
- It is written before the class declaration and after the package statement.
- We use the **import statement in two ways, either import a specific class or import all classes of a particular package.**

For example:

```
import java.util.Scanner; //it imports the Scanner class only
import java.util.*; //it imports all the class of the java.util
package
```

Basic Structure of Java Program

Interface Section

- It is an **optional** section.
- We use the **interface keyword** to create an interface.
- can be used when programmers want to implement **multiple inheritances within a program.**

Example:

```
interface car
{
void start();
void stop();
}
```


Basic Structure of Java Program

Class Definition

- It is **vital part of a Java program**.
- **Without the class, we cannot create any Java program.**
- Classes are the **main and essential** elements of **any Java program**.
- For **defining** a **class** we use “**class**” **keyword** and after that, we write the name of the class you want to give.

Example

```
class first // first is name of class
{
}
```

Basic Structure of Java Program

Main Method Class

- It is essential for **all Java programs**.
 - ▶ Because the execution of all Java programs **starts from the main() method**.
- Inside the main method, we create objects and call the methods.
- We use the following statement to **define the main() method**:

```
} public static void main(String args[])  
{  
  
}
```

Basic Structure of Java Program

Main Method Class

- for Examples:

```
public class Student //class definition  
{  
public static void main(String args[])  
{ //statements  
}  
}
```

Basic Structure of Java Program

Creating, Compiling and Running a Java Program

- To create java program, you will:
 - ▶ **Create a source file**
 - ★ A source file contains **text**, written in the **Java programming language**
 - ★ You can use **any text editor to create and edit source files.**
 - ▶ **Compile the source file into a bytecode file:**
 - ★ The **compiler, javac**, takes **your source file** and translates its text **into instructions** that the **Java Virtual Machine (Java VM)** can **understand.**
 - ★ The **compiler converts** these instructions **into a bytecode file.**

Basic Structure of Java Program

Creating, Compiling and Running a Java Program

- To create java program, you will:
 - ▶ **Run the program contained in the bytecode file**
 - ★ The Java **interpreter** installed on your computer implements the Java VM.
 - ★ **Interpreter** takes your bytecode file and carries out the **instructions** by **translating** them into instructions that **your computer can understand**.

Basic Structure of Java Program

Creating, Compiling and Running a Java Program

- **Note**

- ▶ **Class name must be the same as the file name** where the class lives.
- ▶ Eg. **A class named MyFirstClass** has to be in the **file called MyFirstClass.java**.
- ▶ A program can contain **one or more class definitions** but **only one public class definition**.
- ▶ The program can be **created in any text editor**

Basic Structure of Java Program

Difference between JVM,JRE,JDK

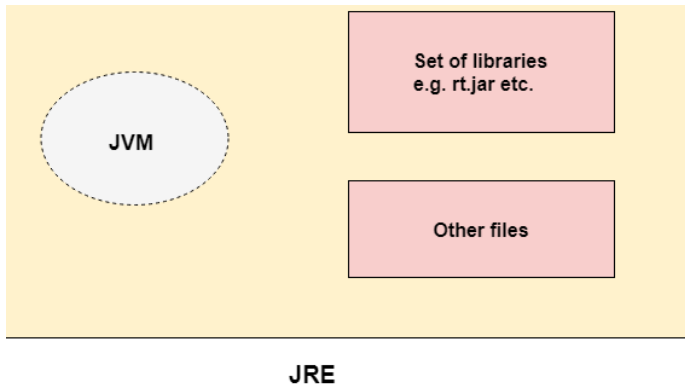
- **JVM (Java Virtual Machine)**
 - ▶ It is a specification that provides a **runtime environment** in which Java bytecode can be executed.
 - ▶ The JVM performs the following main tasks:
 - ★ Loads code
 - ★ Verifies code

Basic Structure of Java Program

Difference between JVM,JRE,JDK

- **JRE(Java Runtime Environment)**

- ▶ It is used to provide the runtime environment.
- ▶ It contains a **set of libraries + other files that JVM uses at runtime.**

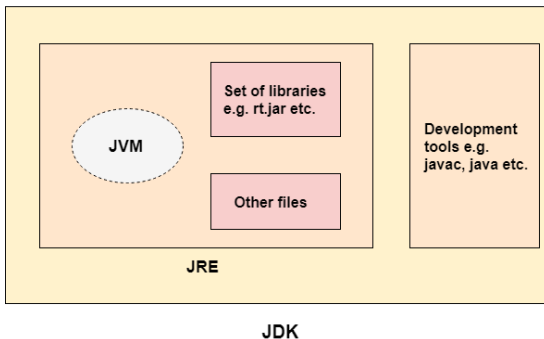


Basic Structure of Java Program

Difference between JVM,JRE,JDK

- **JDK(Java Development Kit)**

- ▶ The JDK contains Java Virtual Machine (JVM) and a few other resources such as
 - ★ a compiler (javac)
 - ★ interpreter/loader (java)
 - ★ an archiver (jar),
 - ★ a documentation generator (Javadoc) and etc...



Basic Structure of Java Program

- Example 1:

```
public class Demo //class definition
{
    public static void main(String args[])
    {

        System.out.println("Hello World");
    }
    //statements
}
```

Basic Structure of Java Program

Class Work

- **Write the program add two numbers. The numbers are 56,45. The class name is add.**
- **Write the program multiplication of the two numbers. The numbers are 7,5 The class name is mul**

Basic Structure of Java Program

- Answer

```
public class add {  
  
    public static void main(String[] args) {  
  
        System.out.println("The two numbers is");  
        int first = 56;  
        int second = 45;  
        System.out.println(first + " " + second);  
        int sum = first + second;  
        System.out.println("The sum is: " + sum);  
    }  
}
```

Basic Structure of Java Program

- Answer

```
public class mul {  
  
    public static void main(String[] args) {  
  
        System.out.println("The two numbers is");  
        int first = 56;  
        int second = 45;  
        System.out.println(first + " " + second);  
        int mul = first * second;  
        System.out.println("The Multiplication is:" +mul);  
    }  
}
```

Basic Structure of Java Program

Class Work

- Swap two numbers using temporary variable

Basic Structure of Java Program

- **Answer**

```
public class s
{
    public static void main(String args[])
    {
        int a=5;
        int b=7;
        int c;
        System.out.println("The number befor swap is ");
        System.out.println("a is\t"+ a);
        System.out.println("b is\t"+ b);
        c=a;
        a=b;
        b=c;
        System.out.println("The number after swap is ");
        System.out.println("a is\t" + a);
        System.out.println("b is\t" + b);}}}
```

Basic Structure of Java Program

Java User Input

- Java Scanner class allows the user to take **input from the console**.
- It belongs to **java.util package**.
- It is used to read the input of primitive types like int, double, long, short, float, and byte.
- It is the **easiest** way to **read input in Java program**.
- **Syntax**
 - ▶ **Scanner sc=new Scanner(System.in);**

Basic Structure of Java Program

Java User Input

- Example

```
import java.util.Scanner;

class Input {
    public static void main(String[] args) {

        Scanner x = new Scanner(System.in);

        System.out.print("Enter an integer: ");
        int number = x.nextInt();
        System.out.println("You entered " + number);

        // closing the scanner object
        x.close();
    }
}
```

Basic Structure of Java Program

Class Work

- 1 Write a program that accepts two integer numbers from the keyboard and display the sum, difference, product, quotient, and the remainder of the two numbers.
- 2 Write a program that will accept a floating number from the keyboard and display the integer part and fraction part of the number.
- 3 Write a program that accepts the score of three students mark from the keyboard and display the average mark.

Basic Structure of Java Program

Atomic elements of java (Tokens)

- **Token** is the smaller individual units **inside a program the compiler recognizes** when building up the program.
- There are **five types of Tokens** in Java:
 - 1 Reserved keywords
 - 2 Identifiers
 - 3 Literals
 - 4 Operators
 - 5 Separators

Atomic elements of java (Tokens)

Reserved keywords

- Reserved words are words that **cannot be used as object or variable names in a Java program** because they're already used by the syntax of the Java programming language.
- **Keywords** are **reserved words** in Java that **serve as a code key**.
- These **keywords** are also known as **reserved keywords** which mean they **cannot be used as a variable name, class, method or any other identifier**.
- **Keywords** are **predefined** which have a **unique meaning** and **functionality in Java programming language**.

Atomic elements of java (Tokens)

• Java Keywords

abstract	do	if	private	this
assert	double	implements	protected	throw
boolean	else	import	public	throws
break	enum	instanceof	return	transient
byte	extends	int	short	true
case	false	interface	static	try
catch	final	long	strictfp	void
char	finally	native	super	volatile
class	float	new	switch	while
const	for	null	synchronized	continue
default	goto	package		

Atomic elements of Java (Tokens)

Identifiers

- **Identifiers** are **programmer given names used to identify classes, methods, variables, objects, packages.**
- Java is **case sensitive** language.
- **Identifier Rules:**
 - ① **Must begin with**
 - ★ Letters
 - ★ Underscore characters (_) or Any currency symbol (e.g \$)
 - ② **Remaining characters**
 - ★ Letters
 - ★ Digits

Atomic elements of Java (Tokens)

- **Which One of the following are valid and invalid identifiers**

- 1 MyVariable
- 2 My Variable
- 3 x
- 4 geeks123
- 5 sum_of_array
- 6 geeks12+r
- 7 \$myvariable
- 8 sum_of_array
- 9 123geeks
- 10 a+c
- 11 variable-2
- 12 sum_&_difference

Atomic elements of Java (Tokens)

- Identifies the identifiers from the following source code

```
public class Test
{
    public static void main(String[] args)
    {
        int a = 20;
    }
}
```


Atomic elements of Java (Tokens)

- In the above java code, we have 5 identifiers namely :
 - ① **Test**: class name
 - ② **main**: method name
 - ③ **String**: predefined class name
 - ④ **args**: variable name
 - ⑤ **a**: variable name.

Atomic elements of Java (Tokens)

Literals

- **Any constant value** which can be **assigned to the variable** is called **literal/constant**.
- **Types of Literals in Java**
 - ▶ **Integer Literal:**
 - ★ Integer literals are basically **a number sequence that doesn't contain any decimal point** in between them.

Atomic elements of Java (Tokens)

- **Example of Integer Literal:**

```
// binary
int binaryNumber = 0b10010;
// octal
int octalNumber = 027;

// decimal
int decNumber = 34;

// hexadecimal
int hexNumber = 0x2F; // 0x represents hexadecimal
// binary
int binNumber = 0b10010; // 0b represents binary
```

Atomic elements of Java (Tokens)

- **Types of Literals in Java**

- ▶ **Floating-point Literals**

- ★ A floating-point literal is a numeric literal that has either a **fractional form** or an **exponential form**.

```
class Main {  
    public static void main(String[] args) {  
  
        float myFloat = 3.4;  
  
        // 3.445*10^2  
        double myDoubleScientific = 3.445e2;  
  
        System.out.println(myFloat);    // prints 3.4  
        System.out.println(myDoubleScientific);    // prints 344.5  
    }  
}
```

Atomic elements of Java (Tokens)

- **Types of Literals in Java**

- ▶ **Character Literals**

- ★ Character literals are **unicode character** enclosed inside **single quotes**.

```
char letter = 'a';
```

Atomic elements of Java (Tokens)

- **Types of Literals in Java**

- ▶ **String literals**

- ★ **string literal** is a sequence of characters **enclosed inside double-quotes**.

```
String str1 = "Java Programming";  
String str2 = "Programiz";
```

Atomic elements of Java (Tokens)

- **Types of Literals in Java**

- ▶ **String literals**

- ★ **string literal** is a sequence of characters **enclosed inside double-quotes**.

```
String str1 = "Java Programming";  
String str2 = "Programiz";
```

Variables and Data Types

- Variable in Java is a data container that stores the data values during Java program execution.
- Variable is **a memory location name of the data.**
- There are three kinds of variables in Java.
 - 1 Local variables
 - 2 Instance variables
 - 3 Static/Class Variable

Variables and Data Types

1. Local Variable

- A **local variable** in Java can be **declared** locally in **methods**, **code blocks**, and **constructors**.

```
public class LocalVariableTest {  
    public void show() {  
        int num = 100; // local variable  
        System.out.println("The number is : " + num);  
    }  
    public static void main(String args[]) {  
        LocalVariableTest test = new LocalVariableTest();  
        test.show();  
    }  
}
```

Variables and Data Types

2.Instance Variable

- An instance variable in Java can be declared **outside a block, method or constructor** but **inside a class**.
- These variables are **created** when the **class object is created** and **destroyed** when the **class object is destroyed**.

```
public class InstanceVariableTest {  
    int num; // instance variable  
    InstanceVariableTest(int n) {  
        num = n;  
    }  
    public void show() {  
        System.out.println("The number is: " + num);  
    }  
    public static void main(String args[]) {  
        InstanceVariableTest test = new InstanceVariableTest(75);  
        test.show();  
    }  
}
```

3.Static/Class Variable

- A **static/class variable** can be defined using the **static keyword**.
- These variables are **declared inside a class** but **outside a method and code block**.
- A **class/static variable** can be **created at the start of the program** and **destroyed at the end of the program**.

```
public class StaticVariableTest {
    int num;
    static int count=0; // static variable
    StaticVariableTest(int n) {
        num = n;
        count ++;
    }
    public void show() {
        System.out.println("The number is: " + num);
    }
    public static void main(String args[]) {
        StaticVariableTest test1 = new StaticVariableTest(75);
        test1.show();
        StaticVariableTest test2 = new StaticVariableTest(90);
        test2.show();
        System.out.println("The total objects  " + count);
    }
}
```

Basic Structure of Java Program

Note

- ① We **can not use** variables **with out** initializing them in Java.
- ② **Class numeric variables** are **initialized to zero** and **Boolean variables** are **initialized to false** if you do not initialize them, where as **local variables will not be initialized**.

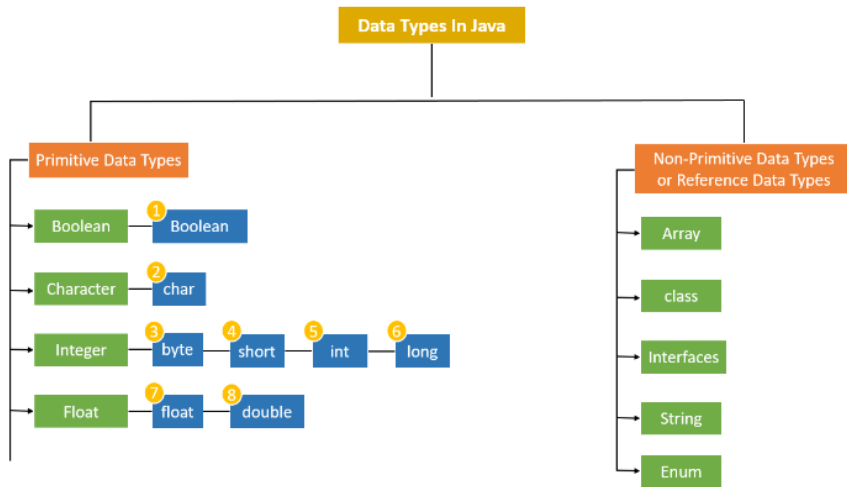
Basic Structure of Java Program

Data Types

- Java has two basic data types
 - 1 Primitive types
 - 2 Reference types

Basic Structure of Java Program

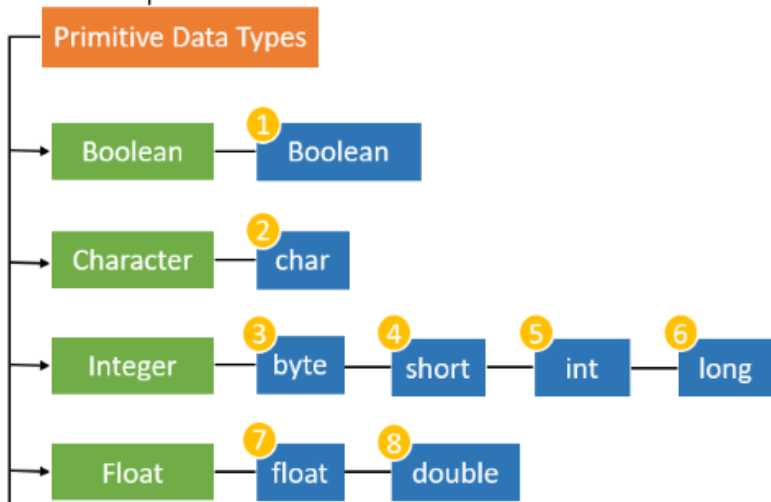
- Java has two basic data types



Basic Structure of Java Program

1. Primitive data types

- Primitive data types **specify the size and type** of variable values.



Primitive data types

Boolean type – Boolean

- A boolean data type can store either True or False.
- The **default** boolean value is **False**.

```
class BooleanDataTypes
{
    public static void main(String args[]) {
        boolean var1 = true;
        if (var1 == true)
        {
            System.out.println("Boolean value is True");
        }
        else
        {
            System.out.println("Boolean value is False");
        }
    }
}
```


Primitive data types

Character type – char

- The char data type stores a single character.
- It stores both **lower case and upper case** characters which must be **enclosed in single quotes**.
- It takes memory space of **16 bits or 2 bytes**.

```
class CharDataType {  
    public static void main(String[] args) {  
        char var1 = 'A';  
        char var2 = 'd';  
        System.out.println(var1);  
        System.out.println(var2);  
    }  
}
```

Primitive data types

Integer type

- An integer type **stores an integer number** with **no fractional or decimal places**.
- There are **four integer types in Java** – byte, short, int, and long.

1. Byte

- ▶ The byte is the **smallest data type among all the integer data types**.
- ▶ It is an 8-bit .
- ▶ **Syntax:**
byte byteVariable;

Primitive data types

Integer type

- **2. Short**

- ▶ short data type is **greater than** byte in terms of **size** and **less than a integer**.
- ▶ The **default size** of this data type: **2 bytes**.

- **3. int data type**

- ▶ The int data type is a 32-bit

- **4. Long**

- ▶ It is **8 bytes(64-bits)** integer data type

Primitive data types

Integer type

- Example

```
class IntegerDataTypes
{
    public static void main(String args[]) {
        int a = 10;
        short s = 2;
        byte b = 6;
        long l = 125362133223L;

        System.out.println("The integer variable is " + a + '\n');
        System.out.println("The short variable is " + s + '\n');
        System.out.println("The byte variable is " + b + '\n');
        System.out.println("The long variable is " + l);
    }
}
```

Primitive data types

Float type

- Floating-point is used for expressions involving **fractional precision**.
- It has **two types**:
 - 1 float
 - 2 double

Primitive data types

Float type

- **1. float**

- ▶ It is a floating-point data type that stores the values including their decimal precision.
- ▶ It is 4 bytes(32-bits) float data type.

- **2. double**

- ▶ It's stored in **64 bits** of memory.
- ▶ This means it represents a **much larger range of possible numbers than float.**

Primitive data types

Integer type

- Example

```
class FloatDataTypes
{
    public static void main(String args[]) {

        float f = 65;
        double d = 876.765;

        System.out.println("The float variable is " + f);
        System.out.println("The double variable is " + d);
    }
}
```

Primitive data types

- Summarize

Data Type	Default Value	Default size	Range
byte	0	1 byte or 8 bits	-128 to 127
short	0	2 bytes or 16 bits	-32,768 to 32,767
int	0	4 bytes or 32 bits	2,147,483,648 to 2,147,483,647
long	0	8 bytes or 64 bits	9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	0.0f	4 bytes or 32 bits	1.4e-045 to 3.4e+038
double	0.0d	8 bytes or 64 bits	4.9e-324 to 1.8e+308
char	'\u0000'	2 bytes or 16 bits	0 to 65536
boolean	FALSE	1 byte or 2 bytes	0 or 1

Basic Structure of Java Program

2. Non-Primitive Datatypes

- Non-Primitive data types **refer to objects** and hence they are called reference types



Type casting

- Type casting is a way of **converting data from one data type to another data type.**
- When you assign the value of one data type to another, you should be aware of the compatibility of the data type.
- There are two types of type casting:
 - 1 Implicit casting(Widening)
 - 2 Explicit casting.(Narrowing)

1. Implicit casting(Widening)

- It happens **automatically** when **converting from a narrower range data type to a wider range data type**.
- It also means converting a lower data type like an int to a higher data type like a double.
- **Implicit casting takes place under two conditions:**
 - 1 **Compatibility** of the **data types**.
 - ★ For example, data types such as **numeric are compatible with other numeric data types**, but they are **not compatible** with boolean, char, or string data types.
 - 2 If the targeted value to be converted has a smaller length.e.g. **4 bytes**, to a larger data type e.g. **8 bytes**.

Type casting

1. Implicit casting(Widening)

- Implicit casting follows the order of conversion as shown below:

```
Byte -> Short -> Char -> Int -> Long -> Float -> Double
```

Type casting

1. Implicit casting(Widening)

- Example

```
public class Main {  
    public static void main(String[] args) {  
        int a = 20;  
        long b = a;    //implicit casting from int to long data type  
        double c = b;  // implicit casting from long to double data type  
  
        System.out.println(a);  
        System.out.println(b);  
        System.out.println(c);  
    }  
}
```

Type casting

1. Implicit casting(Widening)

- Example 2

```
class Main {  
    public static void main(String[] args) {  
        // create int type variable  
        int num = 10;  
        System.out.println("The integer value: " + num);  
  
        // convert into double type  
        double data = num;  
        System.out.println("The double value: " + data);  
    }  
}
```

2. Explicit type casting (Narrowing)

- This type of casting involves assigning a data type of **high range to a lower range**.
- This is done manually as you need to do the casting using the “()” operator.
- Explicit casting follows the order of conversion as shown below:

```
Double -> Float -> Long -> Int -> Char -> Short -> Byte
```

Type casting

2. Explicit type casting (Narrowing)

- Example

```
public class Main {  
    public static void main(String args[]) {  
        double d = 57.17;  
        int i = (int)d;  
        System.out.println(d);  
        System.out.println(i);  
    }  
}
```


Java Operators

- Operators are symbols that perform operations on variables and values.
- For example, `+` is an operator used for addition, while `*` is also an operator used for multiplication.
- **Types of Java Operators:**
 - ▶ Unary Operators
 - ▶ Arithmetic Operators
 - ▶ Assignment Operators
 - ▶ Logical Operators
 - ▶ Relational Operators

Type casting

Unary Operator:

- Unary operator **requires only a single operand**;
- this operator is used to **increment or decrement the value**

Operator	Description
++ (Increment)	This operator is used to increment the value by 1. There are two types of increment, i.e., post-increment (<code>a++</code>) and pre-increment (<code>++a</code>)
- (Decrement).	This operator is used to decrement the value by 1. There are two types of decrement, i.e., post decrement (<code>a--</code>) and pre decrement (<code>--a</code>)
! (Invert)	This operator is used to invert a boolean value (<code>!a</code>).

Unary Operator:

Pre-Increment Operator (++x;)

- If the increment operator (++) is specified before the variable like a prefix (++x), then it is called pre-increment operator.
- In this case, the value of the variable is first incremented by 1, and then further computations are performed.

Unary Operator:

Pre-Increment Operator (++x;)

- Example

```
public class GreatLearning {
    public static void main ( String[] args )
    {
        public class PreIncrementOperator {
            public static void main(String[] args) {
                int variable = 5;
                System.out.println("Original value of the variable = " + variable);
                int preIncrement = ++variable;
                System.out.println("variable = " + variable);
                System.out.println("preIncrement = " + preIncrement);
                System.out.println("++preIncrement = " + ++preIncrement);
            }
        }
    }
}
```

Unary Operator:

Post-Increment Operator (x++;

- If the increment operator (++) is specified after the variable like a postfix (x++), then it is called **post-increment operator**.
- In this case, the original value of the variable (**without increment**) is used for computations and then it is incremented by 1.

Unary Operator:

Post-Increment Operator

- Example

```
public class PostIncrementOperator {  
    public static void main(String[] args) {  
        int variable = 100;  
        System.out.println("Original value of the variable = " + variable);  
        int postIncrement = variable++;  
        System.out.println("postIncrement = " + postIncrement);  
        System.out.println("variable = " + variable + "\n");  
        System.out.println("postIncrement++ = " + postIncrement++);  
        System.out.println("postIncrement++ = " + postIncrement++);  
        System.out.println("postIncrement++ = " + postIncrement++);  
        System.out.println("\npostIncrement = " + postIncrement);  
    }  
}
```

Unary Operator:

Pre-Decrement Operator ($-x$;))

- If the decrement operator ($-$) is mentioned before the variable like a prefix ($-x$), then it is called a pre-decrement operator.
- For this case, the value of the variable is **first decremented by 1**, and then other computations are performed.

Unary Operator:

Pre-Decrement Operator (--x;)

- Example

```
public class PreDecrementOperator {  
    public static void main(String[] args) {  
        int variable = 11;  
        System.out.println( + variable);  
        int preDecrement = --variable;  
        int x=--variable;  
        System.out.println("variable = " + variable);  
        System.out.println("variable = " + x);  
        System.out.println("preDecrement = " + preDecrement);  
        System.out.println("--preDecrement = " + --preDecrement);  
    }  
}
```


Unary Operator:

Post-Decrement Operator (x--)

- If the decrement operator (–) is mentioned after the variable like a postfix (x–), then it is called a **post-decrement operator**.
- For this case, the original value of the variable (without decrement) is used for computations and then it is decremented by 1. .

Unary Operator:

Post-Decrement Operator (x--)

- Example

```
public static void main(String[] args) {  
    int variable = 75;  
    System.out.println("Original value of the variable = " + variable);  
    int postDecrement = variable--;  
    System.out.println("postDecrement = " + postDecrement);  
    System.out.println("variable = " + variable + "\n");  
    System.out.println("postDecrement-- = " + postDecrement--);  
    System.out.println("postDecrement-- = " + postDecrement--);  
    System.out.println("postDecrement-- = " + postDecrement--);  
    System.out.println("\npostDecrement = " + postDecrement);  
}
```

Unary Operator:

Post-Decrement Operator (x--)

- Example

```
class In
{
    public static void main(String args[]) {
        int a = 5;
        int b = 10 + (a++);
        System.out.println(a);
        System.out.println(b);
    }
}
```

Unary Operator:

Post-Decrement Operator (x++);

- Example

```
class Increment {  
    public static void main(String args[]) {  
        int x, y;  
        x = 10;  
        y = x++ + 10;  
        System.out.println("y = " + y);  
  
        System.out.println("After x++: Value of x = " + x);  
    }  
}
```

Unary Operator:

Post-Decrement Operator (x++;

- Example

```
class In
{
    public static void main(String args[]) {
        int a = 5;
        int b = 7;
        int c = a++ + b;
        System.out.println("Post- Increment  c = "+ c);
        int A = 5;
        int B = 7;
        int C = ++A + B;
        System.out.println("Pre- Increment C = "+ C);
        int m = 1, n = 2;
        int o = m++ + n + ++m;
        System.out.println("Example= "+ o);
    }
}
```

Unary Operator:

- Example

```
public class StaticVariableTest {
    int num;
    static int count=0; // static variable
    StaticVariableTest(int n) {
        num = n;
        count ++;
    }
    public void show() {
        System.out.println("The number is: " + num);
    }
    public static void main(String args[]) {
        StaticVariableTest test1 = new StaticVariableTest(75);
        test1.show();
        StaticVariableTest test2 = new StaticVariableTest(90);
        test2.show();
        System.out.println("The total objects  " + count);
    }
}
```

Arithmetic Operators

- Java Arithmetic operators are used to performing fundamental arithmetic operations such as **addition, subtraction, multiplication, and division on numeric data types.**

Operators	Meaning	Description
1. +	Addition or unary plus	Performs addition operation.
2. -	Subtraction or unary minus	Performs subtraction operation.
3. *	Multiplication	Performs multiplication operation.
4. /	Division	Performs division operation.
5. %	Modulo division (Remainder)	Performs remainder after division operation.

Arithmetic Operators

- Example

```
public class IntegerTest {  
    public static void main(String[] args)  
    {  
        // Declaration of instance variables with initialization.  
        int a = 20, b = 10;  
        System.out.println("a: " + a);  
        System.out.println("b: " + b);  
        System.out.println("a + b = " + (a + b));  
        System.out.println("a - b = " + (a - b));  
        System.out.println("a * b = " + (a * b));  
        System.out.println("a / b = " + (a / b));  
        System.out.println("a % b = " + (a % b));  
    }  
}
```


Arithmetic Operators

- Example 2

```
public class Art2
{
    int x = 9; int y = 12; int z = 3;
    void m1()
    {
        int exp1 = x - y / 3 + z * 2 - 1;
        System.out.println("Evaluation1 = " +exp1);
    }
    void m2()
    {
        int exp2 = (x - y)/3 + ((z * 2) - 1);
        System.out.println("Evaluation2 = " +exp2);
    }
    public static void main(String[] args)
    {
        Art2 t = new Art2();
        t.m1();
        t.m2();
    }
}
```

Assignment Operators

- These operators are used to **assign values to a variable**.
- The **left side operand** of the assignment operator is **a variable**, and the **right side operand** of the assignment operator is **a value**.

Assignment Operators

- Example

```
public class Test {  
    public static void main(String args[]) {  
        int a = 10, b = 20, c = 0;  
        c = a + b;  
        System.out.println("c = a + b = " + c );  
        c += a ;  
        System.out.println("c += a = " + c );  
        c -= a ;  
        System.out.println("c -= a = " + c );  
        c *= a ;  
        System.out.println("c *= a = " + c );  
        a = 10; c = 15; c /= a ;  
        System.out.println("c /= a = " + c );  
        a = 10; c = 15; c %= a ;  
        System.out.println("c %= a = " + c );  
    }  
}
```

Logical Operators

- **Logical operators** in Java are those operators which are used to form compound conditions by **combining two or more conditions or relations**.
- In Java, there are **three types of logical operators**.

Operators	Meaning
1. &&	AND operator
2.	OR operator
3. !	NOT operator

Logical Operators

- Example

```
public class Log{  
    public static void main(String args[]){  
        int a=10;  
        int b=5;  
        int c=20;  
        System.out.println(a<b || a<c);  
        System.out.println(a<b&&a<c);  
        System.out.println(a<c || b<c);  
        System.out.println(a<b&&c<a);  
    }  
}
```

Logical Operators

- Example

```
public class Log2 {  
    public static void main(String[] args)  
    {  
        int x = 1;  
        int y = 2;  
        int z = 5;  
        System.out.println("x: " + (!((x+2)==(1+2))));  
        System.out.println("y: " + (!(y==z)));  
        System.out.println("z>x: " + (!(z > x)));  
    }  
}
```

Relational Operators

- **Java Relational Operators** are a bunch of binary operators used to check for relations between two operands, including equality, greater than, less than, etc.

Operator	Operations	Example
<	Less than	<code>x<y</code>
>	Greater than	<code>x>y</code>
<=	Less than or equal to	<code>x<=y</code>
>=	Greater than equal to	<code>x>=y</code>
==	Equal to	<code>x=y</code>
!=	Not equal to	<code>x!=y</code>

Logical Operators

- Example

```
public class test {  
  
    public static void main(String args[]) {  
        int a = 10;  
        int b = 20;  
  
        System.out.println("a == b = " + (a == b) );  
        System.out.println("a != b = " + (a != b) );  
        System.out.println("a > b = " + (a > b) );  
        System.out.println("a < b = " + (a < b) );  
        System.out.println("b >= a = " + (b >= a) );  
        System.out.println("b <= a = " + (b <= a) );  
    }  
}
```


Basic Structure of Java Program

Home Work

- 1 Write a program that reads two integer values from the keyboard and displays the values before and after swapping them **using the third variable**.
- 2 Write a program that reads two integer values from the keyboard and displays the values before and after swapping **without using a third variable**.

Basic Structure of Java Program

Home Work

- ① Write the program to calculate the area of
 - ▶ **Rectangle**
 - ▶ **Circle**
 - ▶ **Triangle**
- ② Write a program that tells whether a number entered **from the keyboard** is an **even** or an **odd** number.
- ③ Write a program that tells whether a number entered from the keyboard is a positive number or a negative number or zero.
- ④ Write a program that tells whether a character entered from the keyboard is in upper case or in lower case or neither.

Basic Structure of Java Program

- **End of Chapter Two**
- **Thank You!!!**